HAMSA VILAS PG MANAGEMENT SYSTEM
API REFERENCE — CURRENT IMPLEMENTATION
==========

Purpose

This document explains the current API in plain English: what each endpoint does, who may call it, which PostgreSQL tables/repositories it uses, and which Android screen consumes it.

Base URLs

Local API: http://127.0.0.1:8080/api/v1
Health check: http://127.0.0.1:8080/health
Production: https://<deployed-host>/api/v1

Authenticated requests use:
  Authorization: Bearer <access_token>
  Content-Type: application/json

Every response includes x-request-id. Provide that ID when reporting a failure. Error responses are JSON and never expose SQL, stack traces, tokens, or provider credentials.

HOW A REQUEST WORKS

Compose screen -> ViewModel -> feature Repository -> Retrofit ApiService -> Axum route -> AuthUser JWT check -> tenant resolver -> application service -> PostgreSQL repository -> JSON response.

For tenant data, every transaction sets the selected schema and app.current_org. Pooled data is protected by both application org filters and PostgreSQL RLS. Siloed organizations use org_<organization-id> instead of pool.

Roles

Owner: all branches and organization data.
Manager: assigned branch only; risky edits/deletes use maker-checker approval.
Tenant: own stay, receipts, complaints, and away periods only.
Platform operator: marketplace listing approval across organizations.
Prospect: authenticated Explore, enquiries, and invitations matching the verified login email. Explore is intentionally available in every role.
Anonymous: health check and DigiLocker provider callback only.

AUTHENTICATION AND BOOTSTRAP

POST /orgs
  Purpose: privileged creation of an organization, first owner, placement, and Main Branch. It is not normal self-registration.
  Header: x-provisioning-key: <PROVISIONING_KEY>
  Request:
    {"name":"Hamsa Vilas","owner_email":"owner@example.com",
     "owner_phone":"+919876500001","owner_name":"Priya Owner"}
  Response:
    {"org_id":"...","owner_id":"..."}
  Tables: public.organizations, public.platform_users, public.user_org_roles, public.tenant_placement, tenant branches.
  Android: no screen. This is a back-office/bootstrap operation.

POST /auth/email/send-otp
  Purpose: send a six-digit email login code. A new email creates an Explorer

```
  account in the configured platform organization. Responses do not reveal
  whether an account existed.
  Request: {"email":"tenant@example.com"}
  Response: {"status":"sent"}
  Internal behavior: lowercases/normalizes email, enforces IP and per-email
  throttles, creates a random code, Argon2-hashes it, stores the hash in
  public.email_otps, then sends through the configured email adapter. The code
  expires after 10 minutes; at most three sends per 15 minutes and a 30-second
  minimum gap are allowed by default. A failed delivery removes the new row so
  it cannot block the next attempt.
  Android: LoginScreen -> LoginViewModel -> AuthRepository.

POST /auth/email/verify-otp
  Purpose: consume the newest usable email code and issue the application's
  access and rotating refresh tokens.
  Request: {"email":"tenant@example.com","code":"123456"}
  Response:
    {"access_token":"<jwt>","refresh_token":"<opaque-token>",
     "is_verified":true,"is_new_device":false}
  Internal behavior: locks the challenge row, performs one Argon2 verification,
  atomically increments attempts or consumes it, marks the email verified, and
  issues a session. Wrong, expired, exhausted, and nonexistent codes return the
  same safe response and perform comparable password-hash work. Five failed
  attempts invalidate a challenge.
  Tables: public.email_otps, public.platform_users, public.user_org_roles,
  public.refresh_tokens.

POST /auth/refresh
  Purpose: rotate the refresh token and return a new access/refresh pair.
  Request: {"refresh_token":"<refresh-token>"}
  Response: {"access_token":"<new-access-jwt>","refresh_token":"<new-token>"}
  Internal behavior: consumes the old refresh hash, re-reads role/membership,
  stores the new hash, and rejects replayed or expired tokens.
  Android: TokenAuthenticator, using the bare OkHttp client.

POST /auth/switch-organization
  Purpose: switch the same signed-in account between active membership
  contexts, for example platform/prospect for Explore and PG A/tenant for
  private resident features.
  Request:
    {"organization_id":"<org-id>","role":"tenant",
     "refresh_token":"<current-refresh-token>"}
  Response: {"access_token":"<new-jwt>","refresh_token":"<new-token>"}
  Internal behavior: confirms the requested membership is active, rotates the
  refresh token, and issues claims for exactly that organization and role.
  Tables: public.user_org_roles, public.organizations, public.refresh_tokens.
  Android: ProfileScreen membership selector -> AuthRepository.

POST /auth/logout
  Purpose: revoke every refresh token for the caller and revoke the current
  access-token JTI in the process blocklist.
  Response: {"status":"logged_out"}
  Android: AppViewModel -> AuthRepository -> SessionManager.

POST /auth/google
  Purpose: sign in an already-bound Google identity. It does not auto-create a
  user. Request: {"id_token":"<google-id-token>"}. Tables:
  public.platform_users and public.user_org_roles.

POST /auth/verify-identity
  Purpose: tenant first-login identity gate.
  Request:
```

{"method":"digilocker","provider_ref":"provider-ref","phone":"9876500001","emergency_contact":"9000000000","country":"IN"}
  Response: {"status":"verified","reason":null}
  Tables: public.verifications and platform_users.verified.
  Android: VerifyIdentityScreen / VerifyIdentityViewModel.

CURRENT USER

GET /me
  Returns: {"user_id":"...","org_id":"...","role":"manager",
  "branch_id":"...","full_name":"Manoj","verified":true}.
  Tables: public.platform_users and public.user_org_roles.
  Android: AuthRepository.completeLogin() and SessionManager.

GET /me/memberships
  Returns every active context for the signed-in user. Pending and moved-out
  memberships are deliberately omitted.
  Response:
    [{"membership_id":"...","organization_id":"...",
      "organization_name":"Main Branch PG","role":"tenant",
      "branch_id":"..."}]
  Query: joins public.user_org_roles, organizations, and platform_users where
  membership_status='active'. Android: ProfileScreen context switcher.

GET /me/tenant-invitations
  Returns invitations whose normalized email equals the caller's verified
  login email. It is independent of the currently selected PG context.
  Tables: public.tenant_invitations, public.platform_users.
  Android: InvitationListScreen.

GET /me/ledger
  Tenant-only own balance: rent, deposit, paid, accrued, outstanding.
  Query: resolve residents by user_id, then aggregate payments through billing.
  Tables: tenant residents, payments, receipts.
  Android: TenantHomeScreen.

GET /me/receipts
  Tenant-only receipt metadata.
  Tables: tenant residents and receipts.
  Android: TenantHomeScreen and TenantReceiptsScreen.

GET /me/away-periods
  Tenant-only own away periods. Tables: tenant away_periods and residents.
  Android: AwayScreen.

GET /me/receipts/{id}/download
  Tenant-only receipt download. Verifies ownership, then streams a local PDF or
  redirects to a short-lived object-storage URL.

TENANT INVITATIONS AND ONBOARDING

Invitation status values
  invited, email_verified, kyc_pending, kyc_verified, kyc_failed,
  approval_pending, approved, rejected, cancelled, expired, active.
  Invalid lifecycle transitions return 409 Conflict.

GET /tenant-invitations
  Owner: every invitation in the selected organization.
  Manager: invitations in the manager's active branch membership only.
  Other roles: 403.

Query: public.tenant_invitations WHERE org_id=$selected_org, then manager branch scope. Android: InvitationManagementScreen.

POST /tenant-invitations
Owner or correctly scoped manager creates an invitation.
Request:

```
{
  "email":"tenant@example.com",
  "full_name":"Tenant Person",
  "branch_id":"<branch-id>",
  "intended_room_id":"<optional-room-id>",
  "intended_bed_id":"<optional-bed-id>",
  "monthly_rent_paise":900000,
  "deposit_paise":1800000,
  "planned_move_in_date":"2026-09-01",
  "require_identity_verification":true,
  "expires_in_days":7
}
```

Response: the complete invitation record, including status and timestamps.
Internal behavior: validates staff/organization/branch scope, normalizes the email, validates amounts and expiry, inserts public.tenant_invitations, then sends the dedicated invitation email. The link contains only invitation ID.
Android: CreateInvitationScreen.

GET /tenant-invitations/{id}
Owner can read same-organization records; manager only same branch; invited user only when their verified email matches. Platform operator is denied. Cross-organization IDs return 403 or a safe 404.
Android: InvitationDetailScreen and InvitationReviewScreen.

POST /tenant-invitations/{id}/accept
Invited user only. The server obtains email from the verified account; there is no email field in the request. An empty request body is used.
Response: {"status":"kyc_pending"} when KYC is required, otherwise {"status":"approval_pending"}. Expired or already-used invitations return 409. Tables: tenant_invitations, platform_users.

POST /tenant-invitations/{id}/decline
Invited user only; moves an eligible invitation to cancelled.
Response: {"status":"cancelled"}.

POST /tenant-invitations/{id}/approve
Owner or correctly scoped manager; self-approval is forbidden.
Response: {"status":"active"}.
Internal transaction:

1. SELECT ... FOR UPDATE locks the invitation.
2. Rechecks status, accepted user, verified email, org/branch, room/bed, required KYC, and bed availability.
3. Creates or activates public.user_org_roles membership.
4. Creates or links the tenant-schema residents row and optional bed.
5. Marks invitation active and writes tenant audit_log.
6. Commits all writes together. Any failure rolls everything back.

Tables: public.tenant_invitations, platform_users, user_org_roles, invitation_identity_verifications; tenant branches, rooms, beds, residents, audit_log. Android: InvitationReviewScreen.

POST /tenant-invitations/{id}/reject
Owner or correctly scoped manager; moves approval-eligible invitation to rejected and sends the rejection notification.

POST /tenant-invitations/{id}/cancel
Owner or correctly scoped manager; cancels an eligible invitation.

```
POST /tenant-invitations/{id}/resend
  Extends expiry by seven days and sends a fresh notification after the
  configured cooldown. Refuses approved, active, cancelled, rejected, and
  expired records. Response: refreshed invitation record.

GET /tenant-invitations/{id}/onboarding-status
  Same visibility as invitation detail.
  Response:
    {"invitation_id":"...","status":"approval_pending",
     "require_identity_verification":true,
     "next_action":"wait_for_approval"}
  next_action values: accept_or_decline, continue_onboarding,
  complete_identity_verification, contact_property, wait_for_approval,
  activation_in_progress, open_tenant_home, none.
  Android: TenantOnboardingStatusScreen polls this while visible.

POST /tenant-invitations/{id}/digilocker/start
  Invited user with matching verified email. Creates short-lived OAuth state,
  nonce, and PKCE verifier; stores state/nonce hashes and encrypted verifier.
  Response: {"authorization_url":"https://..."}.
  Tables: public.digilocker_oauth_sessions. Android opens Chrome Custom Tabs.

GET /integrations/digilocker/callback?state=...&code=...
  Public OAuth redirect called by DigiLocker, not directly by Android. Validates
  state, expiry, one-time consumption, provider response, and nonce; exchanges
  the code only on the backend; records the minimal result; transitions KYC;
  then redirects to hamsavilas://onboarding?invitation_id=...&status=....
  Provider access tokens and documents are never sent to Android or stored.

GET /tenant-invitations/{id}/verification
  Returns null before a result, or:
    {"provider":"digilocker","provider_reference":"safe-reference",
     "document_type":"aadhaar","status":"verified",
     "verified_at":"...","safe_failure_reason":null}
  Tables: public.invitation_identity_verifications.
  Android: DigiLockerVerificationScreen.

POST /tenant-invitations/{id}/manual-review
  Owner or correctly scoped manager when configuration permits manual review.
  Request: {"decision":"approved","reason":"Government ID checked in person"}
  Response: {"status":"approval_pending"}; rejected decision returns
  {"status":"kyc_failed"}. Records reviewer, decision, reason, request ID,
  and timestamp in public.manual_kyc_reviews.

BRANCHES, ROOMS, BEDS, RESIDENTS
--------------------------------

GET /branches
  Owner: all active branches. Manager: assigned branch only. Tenant: 403.
  Query: tenant branches WHERE org_id=$org AND deleted_at IS NULL.
  Response: [{"id":"...","name":"Main Branch"}]
  Android: BranchesScreen and shared branch pickers in Dashboard, Occupancy,
  and Payments.

POST /branches
  Creates a branch and reconciles configured onboarding rooms/beds.
  Request: {"name":"Koramangala"}
  Response: {"status":"created"}
  Tables: tenant branches, audit_log.

PATCH /branches/{id} and DELETE /branches/{id}
```

```
  Update or soft-delete a branch. Owner changes apply; manager changes create a
  pending change request. Tables: branches, change_requests, audit_log.
  PATCH request: {"name":"HSR Layout","address":"12 Example Road"}
  Response: {"status":"applied","change_request_id":null} or status pending.

GET /branches/{branch_id}/rooms
  Active rooms for a branch.
  Query shape: SELECT id,branch_id,number,cooling_type FROM rooms WHERE
  org_id=$org AND branch_id=$branch AND deleted_at IS NULL.
  Response: [{"id":"...","branch_id":"...","number":"101","cooling_type":"ac"}]
  Android: RoomsScreen, Occupancy, Payments floor/room filters.

POST /branches/{branch_id}/rooms
  Request: {"number":"101","cooling_type":"ac"}
  Response: {"id":"<room-id>"}
  Tables: rooms and audit_log.

PATCH /rooms/{id} and DELETE /rooms/{id}
  Update/soft-delete room with the same maker-checker behavior as branches.
  Tables: rooms, change_requests, audit_log. Android: RoomsScreen/Occupancy.

GET /rooms/{room_id}/beds
  Beds in one room, joined to active residents for occupant ID/name.
  Tables: beds, residents. Android: room expansion in Occupancy and Rooms.

GET /branches/{branch_id}/beds
  All branch beds in one batch (replaced the old room-by-room fan-out).
  Tables: beds, rooms, active residents. Android: Payments and Rooms.

POST /rooms/{room_id}/beds
  Request: {"label":"B"}
  Response: {"id":"<bed-id>"}
  Tables: beds and audit_log. Android: add-bed dialogs.

GET /branches/{branch_id}/residents
  Active roster for the branch. Sensitive phone/rent/deposit fields are not
  returned. Query filters residents by org, branch, and deleted_at.
  Response: [{"id":"...","branch_id":"...","full_name":"Raj Tenant","status":"ac
tive","plan_kind":"monthly"}]
  Android: BranchDetailScreen / ResidentsViewModel.

POST /branches/{branch_id}/residents
  Creates an operational resident record. Phone is optional contact data and
  never creates login credentials or tenant membership. Use the invitation
  flow for account onboarding and private access.
  Request:
    {"full_name":"Raj Tenant","phone":"9876500001","plan_kind":"monthly","rent_p
aise":500000,"deposit_paise":1000000}
  Response: {"id":"<resident-id>"}
  Tables: tenant residents; public platform_users and user_org_roles; audit_log.

PATCH /residents/{id} and DELETE /residents/{id}
  Update/soft-delete resident. Manager changes become approvals; owner changes
  apply. Tables: residents, change_requests, audit_log.
  Android: Residents and ResidentDetailScreen.

POST /residents/{id}/link-user
  Links an existing platform identity to a resident. Used for prospect -> tenant
  promotion. Updates residents.user_id after org/role checks.

POST /residents/{id}/assign-bed
  Assigns or clears a bed. Validates same organization, branch, room, and free
```

bed. Tables: residents, beds, rooms, audit_log.

PAYMENTS AND FINANCE

POST /residents/{id}/payments
Records cash, calculates the balance, creates a receipt, renders a PDF, and returns IDs. Use Idempotency-Key for safe retry.
Header: Idempotency-Key: payment-2024-07-0001
Request: {"amount_paise":500000}
Response: {"payment_id":"...","receipt_id":"...","receipt_number":"HV-0001"}
Tables: payments, receipts, residents, public receipt_templates, object store, audit_log. Android: ResidentDetail payment action.

GET /residents/{id}/ledger
Selected resident's rent/deposit/accrued/paid/outstanding. Tables: residents, payments, billing engine. Android: ResidentDetail and Occupancy sheet.

GET /residents/{id}/receipts
Selected resident's receipt metadata. Tables: residents, receipts.
Android: ResidentDetail.

GET /receipts/{id}/download
Owner/manager receipt download after org, resident, and ready-state checks. Reads storage_key from receipts; streams or redirects to object storage.

GET /branches/{id}/dues
Batch resident due rollup. Reads residents and payments and calculates accrued and outstanding values. Android: Payments; avoids one ledger request per user.

GET /finance/branch/{id}
Branch totals: collected, deposits, accrued, outstanding, residents, beds, and occupancy. Tables: residents, payments, beds, rooms.

GET /finance/org
Owner-only total plus one report per branch. Reads the same finance tables.
Response shape:
{"total":{"collected_paise":0,"deposits_paise":0,"accrued_paise":0,"outstanding_paise":0,"residents_active":0,"residents_total":0,"beds_total":0,"beds_occupied":0},"branches":[]}

GET /org/receipt-template and PUT /org/receipt-template
Per-organization receipt branding. GET uses saved row or default; PUT is owner-only. Table: public.receipt_templates. Android: TemplateScreen.
PUT request: {"display_name":"Hamsa Vilas","address":"Bengaluru","footer":"Thank you","accent":"#1464D2","logo_key":null}

POST /residents/{id}/move-out
Creates settlement, deductions, refund/forfeit result and audit event.
Request: {"last_day":"2024-07-31","notice_days":30,"deductions":[{"reason":"Broken key","amount_paise":5000}]}
Tables: move_out_settlements, move_out_deductions, residents, audit_log.

COMPLAINTS, NOTICES, AWAY, APPROVALS

GET /complaints
Tenant own; manager branch non-business; owner organization. Tables: complaints, complaint_events, residents/users for visibility.
Android: ComplaintsScreen.

POST /complaints

  Request: {"title":"Water issue","description":"No hot water","business_level":
false,"branch_id":"..."}
  Creates complaint and first complaint_events row; audits the write.

POST /complaints/{id}/transition
  Request: {"to":"in_progress"}
  Validates role, ownership/branch, and state-machine transition, then writes a
  complaint_events row.

GET /notices
  Branch and organization-wide notices visible to caller. Tables: notices,
  residents for tenant branch visibility. Android: NoticesScreen.

POST /notices
  Request: {"title":"Rent due reminder","body":"Due on the 5th","branch_id":"...
"}
  Inserts notices, writes audit, and calls the notifier adapter.

GET /away-periods
  Staff view of active away periods for permitted branch/organization. Tables:
  away_periods and residents. Android: staff Away view.

POST /away-periods
  Tenant report. Request: {"from_date":"2024-07-20","to_date":"2024-07-25","note
":"Family visit"}
  Tables: away_periods, residents, audit_log. Android: AwayScreen.

GET /me/away-periods and DELETE /me/away-periods/{id}
  Read/cancel only the signed-in tenant's own period. Ownership is resolved by
  residents.user_id, not by a client-supplied resident ID.

GET /change-requests
  Owner pending approval queue. Query filters change_requests.status='pending'.
  Android: ApprovalsScreen.

POST /change-requests/{id}/approve or /reject
  Owner-only decision. Approval applies the stored operation and audits in one
  transaction; rejection leaves the target unchanged.

MARKETPLACE AND ENQUIRIES

GET /discover/listings?limit=50
  Authenticated live-listing feed available to prospect, tenant, owner, and
  manager contexts. Reads only public.public_listings joined to
  public.listing_vacancy; stale/non-live listings are excluded.

GET /discover/search?q=Koramangala&city=Bengaluru&limit=20
  Authenticated text/city search over the same public projection. It never reads
  operational residents, beds, rooms, or phone numbers.

GET /discover/nearby?lat=12.9716&lng=77.5946&limit=20
  Anonymous nearest-first listing search using public listing coordinates.

GET /discover/listings/{branch_id}
  Returns one live public listing or 404. A branch ID here does not grant access
  to operational branch data. Android: ExploreScreen and ExploreViewModel.

PUT /listings/{branch_id}
  Owner/manager creates or updates a marketing draft in public.public_listings.
  Request:
    {"title":"Hamsa Vilas Koramangala","area":"Koramangala","city":"Bengaluru","

```
lat":12.9352,"lng":77.6245,"description":"Furnished PG","amenities":"Wi-Fi, meal
s"}

POST /listings/{branch_id}/submit
  Moves the listing draft to operator review. Table: public.public_listings.

GET /listings
  Owner/manager organization listing statuses. Table:
  public.public_listings filtered by org_id. Android: MyListingsScreen.

GET /operator/listings
  Platform-operator pending queue across organizations. Table:
  public.public_listings with pending status.

POST /operator/listings/{branch_id}/publish
POST /operator/listings/{branch_id}/reject
POST /operator/listings/{branch_id}/unlist
  Platform-operator lifecycle operations on public.public_listings.

POST /listings/{branch_id}/enquire
  Requires email-OTP authentication and creates a lead using the caller's
  stable user ID. It does not ask for another OTP.
  Request: {"note":"Need one bed"}
  Response: {"status":"received"}
  Tables: public.platform_users and public.enquiries. The legacy phone column
  stores the non-sensitive marker "email-authenticated" until a later schema
  cleanup; it is not used for identity.

GET /enquiries
  Manager sees own branch leads; owner sees all organization leads.
  Android: EnquiriesScreen.

POST /enquiries/{id}/status
  Request: {"status":"contacted"}
  Updates public.enquiries after branch/organization authorization.

DEVICES, AUDIT, ADMIN, HEALTH
-----------------------------

POST /me/devices
  Registers an FCM device token in public.device_tokens for the signed-in user.
  Request: {"token":"<fcm-token>","platform":"android"}
  Android: app bootstrap notification registration.

DELETE /me/devices/{token}
  Removes only the caller's matching device token.

GET /audit?limit=50
  Owner-only audit feed, maximum 500 rows. Query:
  SELECT ... FROM audit_log WHERE org_id=$org ORDER BY id DESC LIMIT $limit.
  Table: tenant audit_log. Android: AuditScreen.

POST /admin/archive
  Owner-only maintenance action for soft-deleted rows older than a cutoff.
  Writes JSON to object storage, records archive_manifest, and purges hot rows.
  Request: {"older_than_days":30}
  Table: tenant soft-deleted tables and archive_manifest.

POST /admin/promote
  Owner-only pooled-to-silo migration. Copies this organization's tenant tables
  from pool to org_<id>, switches public.tenant_placement, invalidates the
  resolver cache, and removes that organization's pool rows transactionally.
```

```
  Response: {"status":"promoted","mode":"siloed","schema":"org_<id>"}

PLATFORM ADMIN DIAGNOSTICS
--------------------------
These endpoints are separate from the owner maintenance endpoints above.
They require an authenticated user whose JWT role is platform_operator.
Owners, managers, tenants, prospects, and anonymous callers receive 403.
They are designed for safe operational debugging: no request body, password,
OTP, bearer token, refresh token, phone number, email address, or SQL text is
returned.

GET /platform/admin/system/health
  Purpose: platform-only readiness view. Shows whether the API currently has
  its database dependencies available and includes the request ID.
  Response:
    {"status":"ok","database_ready":true,"request_id":"..."}
  Tables: none. It reads in-process application state.
  Android: no current mobile screen. Intended for a future platform-admin UI
  or secure operations tooling.

GET /platform/admin/organizations
  Purpose: inventory of active organization IDs for platform support.
  Response:
    {"organizations":["org-id-1","org-id-2"],"count":2}
  Query behavior:
    SELECT id FROM public.organizations
    WHERE deleted_at IS NULL ORDER BY created_at.
  Tables: public.organizations. No organization names or user PII are returned.
  Android: no current mobile screen; future platform-admin UI only.

GET /platform/admin/logs
  Purpose: search safe, structured request logs captured by API middleware.
  Query parameters (all optional):
    level=INFO|WARN|ERROR
    status_code=200|401|403|500
    route=/api/v1/payments
    page=1 (minimum 1)
    limit=50 (minimum 1, maximum 100)
  Example:
    /platform/admin/logs?level=ERROR&status_code=500&page=1&limit=50
  Response:
    {
      "items":[{
        "timestamp":"2026-08-11T10:30:00Z",
        "level":"ERROR",
        "event_type":"request_completed",
        "route":"/api/v1/payments",
        "method":"GET",
        "status_code":500,
        "latency_ms":842,
        "request_id":"request-id"
      }],
      "page":1,"limit":50,"total":1
    }
  Internal behavior: every completed request is written to the existing
  structured tracing output and to a bounded in-process ring buffer. The
  buffer holds at most 10,000 latest records and drops the oldest record when
  full. Logs reset when the server restarts; they are not yet a historical
  PostgreSQL audit table.
  Tables: none. Android: future platform-admin UI only.

GET /platform/admin/logs/summary
```

```
  Purpose: quick operational view of the currently retained request logs.
  Response:
    {"total":124,"errors":3,"warnings":9,"average_latency_ms":142}
  Internal behavior: calculates counts of 5xx and 4xx responses and average
  latency from the same safe in-process log buffer. It does not inspect
  request/response bodies.
  Tables: none. Android: future platform-admin UI only.

GET /health
  Unauthenticated readiness check outside /api/v1.
  Response: {"status":"ok","db_ready":true,"request_id":"..."}

ANDROID SCREEN MAP
------------------

Login: /auth/email/send-otp, /auth/email/verify-otp, /me
Invitation List: /me/tenant-invitations
Invitation Detail: invitation get/accept/decline/onboarding-status
Tenant Onboarding Status: invitation onboarding-status/verification
DigiLocker Verification: digilocker/start, verification; callback via deep link
Invitation Management: tenant-invitations list/get/resend/cancel
Create Invitation: tenant-invitations create; branches/rooms/beds pickers
Invitation Review: invitation get/manual-review/approve/reject
Tenant Home: /me/ledger, /me/receipts
Tenant Receipts: /me/receipts, /me/receipts/{id}/download
Profile: /me, /me/memberships, /auth/switch-organization, logout
Branches: /branches and branch CRUD
Branch Detail/Residents: branch residents, resident CRUD, listing submit
Occupancy: branches, rooms, beds, resident registration/assignment
Rooms & Beds: rooms, room beds, resident placement
Payments: branches, rooms, branch beds, residents, batch dues, payment
Resident Detail: ledger, receipts, payment, move-out, edit/delete
Complaints: complaint list/create/transition
Notices: notice list/create
Away: own/report/cancel away periods
Approvals: change request list/approve/reject
Audit: audit list
Template: receipt template get/update
Explore: authenticated discovery/search/nearby/detail and authenticated enquiry
Enquiries: enquiry inbox/status
My Listings: listing draft/submit/list

USEFUL CURL EXAMPLES
--------------------

curl -i http://127.0.0.1:8080/health

curl -sS "$API_BASE/branches/$BRANCH_ID/residents" \
  -H "Authorization: Bearer $ACCESS_TOKEN"

curl -sS -X POST "$API_BASE/residents/$RESIDENT_ID/payments" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: payment-2024-07-0001' \
  -d '{"amount_paise":500000}'

curl -sS "$API_BASE/discover/search?q=Koramangala&limit=20" \
  -H "Authorization: Bearer $ACCESS_TOKEN"

SOURCE REFERENCES
-----------------
HTTP routes: crates/api/src/routes/
```

JWT extractor: crates/api/src/extractors.rs
Middleware: crates/api/src/middleware/
Use cases and ports: crates/application/src/
SQL repositories: crates/infrastructure/src/persistence/
Control schema: migrations/control_plane/0001_init.sql
Tenant schema: migrations/tenant/0001_init.sql
Retrofit contract: ../android_kotlin/app/src/main/java/com/hamsavilas/pg/core/network/ApiService.kt
Android features: ../android_kotlin/app/src/main/java/com/hamsavilas/pg/feature/

If code and this reference disagree, the running route and its tests are the source of truth. Update this document whenever an API contract or authorization rule changes.